



VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



5

CODE OPTIMIZATION

Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

Optimization of Basic Blocks

- Optimization is applied to the basic blocks after the intermediate code generation phase of the compiler.
- Optimization is the process of transforming a program that improves the code by *consuming fewer resources* and *delivering high speed*.

1. The DAG Representation of Basic Blocks

- DAG representation or Directed Acyclic Graph representation is used to *represent the structure of basic blocks*.
- Its main aim is to *perform the transformation on basic blocks*.
- The **leaf nodes** of the directed acyclic graph represent a *unique identifier* that can be a variable or a constant.
- The **non-leaf nodes** represent an *operator symbol*.
- The nodes are also given a string of identifiers to use as labels for the computed value.

Algorithm: Construction of DAG

INPUT : The input will contain a basic block.

OUTPUT: A DAG for the basic block containing the following information

1. Each node of the graph represents a label.
 - If the node is a **leaf node**, the label represents an **identifier**.
 - If the node is a **non-leaf node**, the label represents an **operator**.
2. Each node contains a list of attached identifiers to hold the computed value.

Case 1:

x = y op z // x, y, and z are operands and op is an operator.

Case 2:

x = op y // x and y are operands and op is an operator.

Case 3:

x = y // x and y are operands.

METHOD

- Steps to draw a DAG handling the above three cases.

Step 1:

- If the y operand is not defined, then create a **node (y)**.
- If the z operand is not defined, create a node as **node(z)** for case(1)

Step 2:

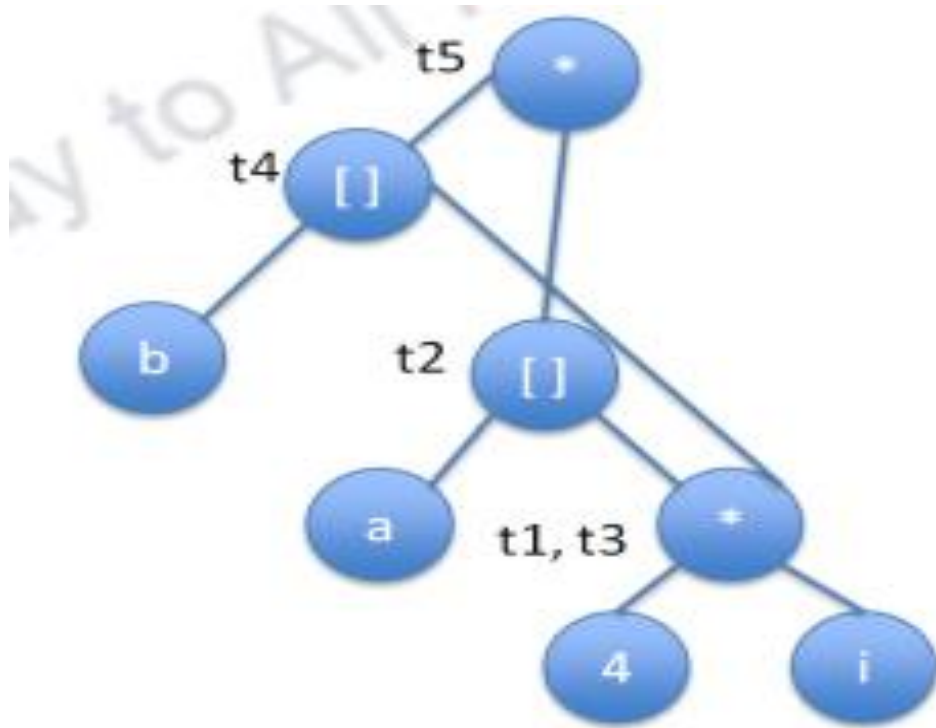
- For case 1: Create a **node(op)** with **node(y)** as its left child and **node(z)** as its right child. Let the name of this node be n.
- For case 2: Check whether there is a **node(op)** with one child node as **node(y)**. If there is no such node, then create a node.
- For case 3 : node n will be **node(y)**.

Step 3:

- For a node(x), delete x from the list of identifiers.
- Add x to the list of attached identifiers list found in step 2. At last, set node(x) to n.

- Construct DAG for the given three-address code instructions:

```
t1 := 4 * i  
t2 := a[t1]  
t3 := 4 * i  
t4 := b[t3]  
t5 := t2 * t4
```

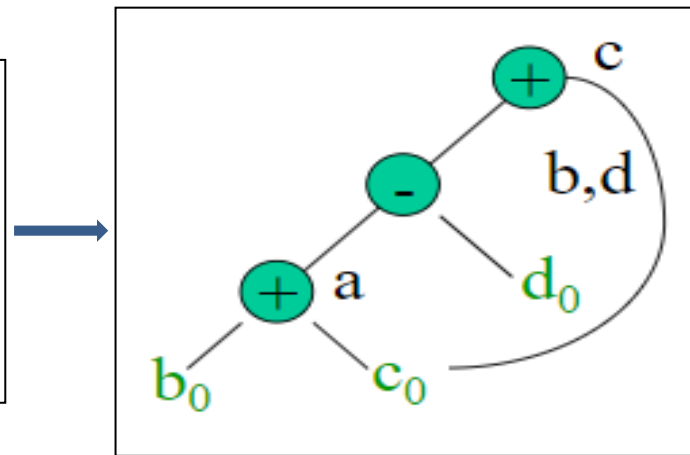


- Instruction 1, checks if the constant '4' and 'i' are available. As it is not available, we create these two nodes.
- Since, these are new nodes, a new internal node with operator, '*' is created with left and right children as '4' and 'i' respectively and the interior node is labeled as t1.
- For line 2, the nodes are created with 'a' as the left child and since t1 is already there it is used and a new interior node is created and is labeled as t2.
- For the instruction at line 3, since nodes '4', 'i' and '*' whose left and right children are '4' and 'i' are already there, they are used and thus t1 and t3 corresponds to the same node.

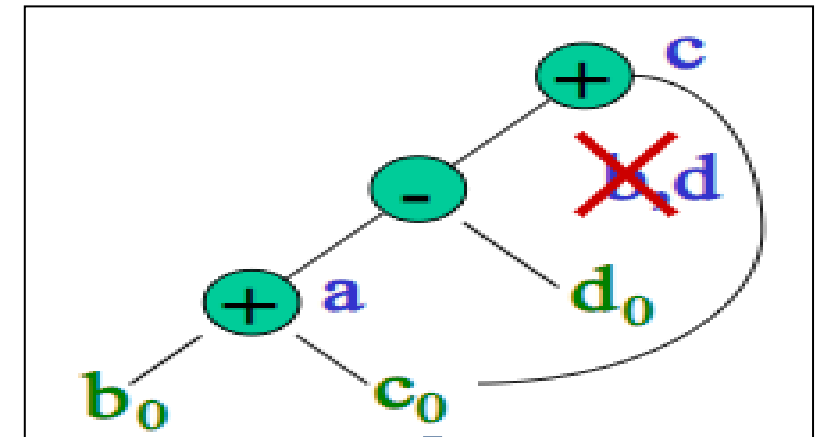
a. Finding Local Common Subexpressions

- Common subexpressions can be detected by noticing, as a new node M is about to be added, whether there is an existing node N with the same children, in the same order, and with the same operator.
- This technique was introduced as the "value-number" method of detecting common subexpressions
- DAG (Directed Acyclic Graph) is used to eliminate common subexpressions.
- A DAG for the block

```
a := b + c
b := a - d
c := b + c
d := a - d
```



```
a := b + c
b := a - d
c := b + c
d := b
```

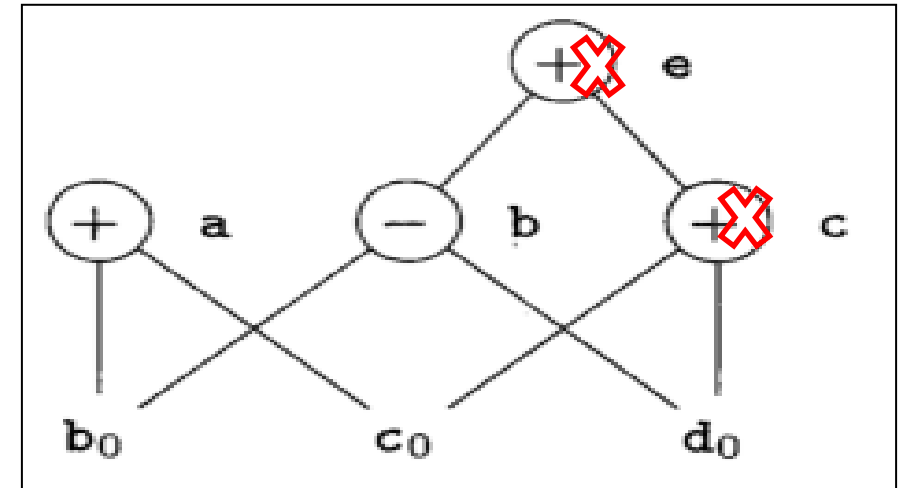
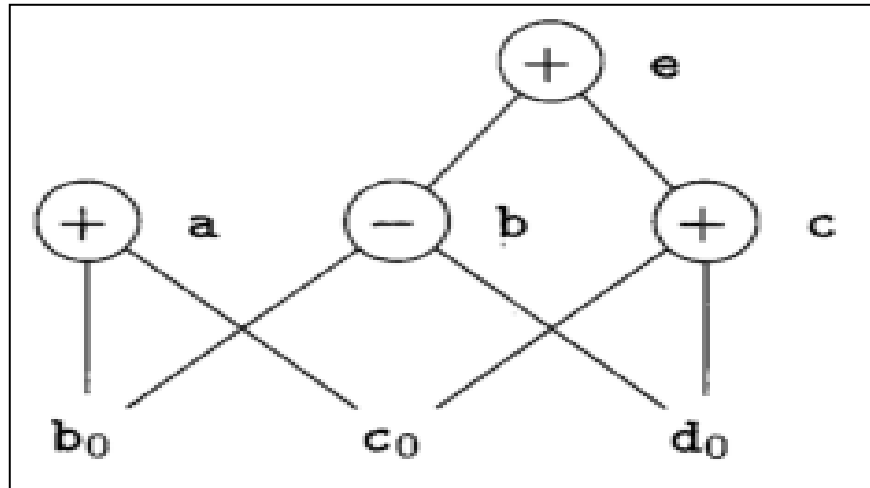


```
a := b + c
d := a - d
c := d + c
```

b. Dead Code Elimination

- Dead code is defined as that part of the *code that never executes* during the program execution.
- Delete from a DAG any root (node with no ancestors) that has no live variables attached
- Repeat until we remove all nodes from the DAG that correspond to dead code.

a = b + c
b = b - d
c = c + d
e = b + c



- The two occurrences of the sub-expressions $b + c$ computes the same value.
- Value computed by a and e are the same.
- Assume a and b are live out.
- Remove first e and then c . a and b remain.

c. The Use of Algebraic Identities (Algebraic Transformations)

- Algebraic transformations can be used to *change the set of expressions computed by a basic block* into an algebraically equivalent set.

Algebraic identities

- Algebraic identities represent another important class of optimizations on basic blocks.

$$\begin{aligned}x + 0 &= 0 + x = x \\x - 0 &= x \\x + 1 &= 1 + x = x \\x / 1 &= x\end{aligned}$$

Eliminate computations from a basic block

Reduction in strength

- Replace expensive statement/ instruction with cheaper ones.

$$\begin{aligned}x ** 2 &= x * x \\2.0 * x &= x + x \\x / 2 &= x + 0.5\end{aligned}$$

$$\begin{aligned}x \uparrow 2 &= x * x \\2 * x &= x + x \\x * 2 &= x \ll 1 \\x * 4 &= x \ll 2 \\x * 8 &= x \ll 3 \\x / 2 &= x \gg 1\end{aligned}$$

Constant folding

- The constant expressions are evaluated at the compile time and the constant expressions are replaced by their values.

$$2 * 3.14 = 6.28$$

Associative laws

- Associative laws may also be applied to expose common sub expressions.
- If the source code has the assignments

```
a = b + c;  
e = c + d + b;
```

Intermediate code

```
a = b + c  
t = c + d  
e = t + b
```

If t is not needed
outside this block

```
a = b + c  
e = a + d
```

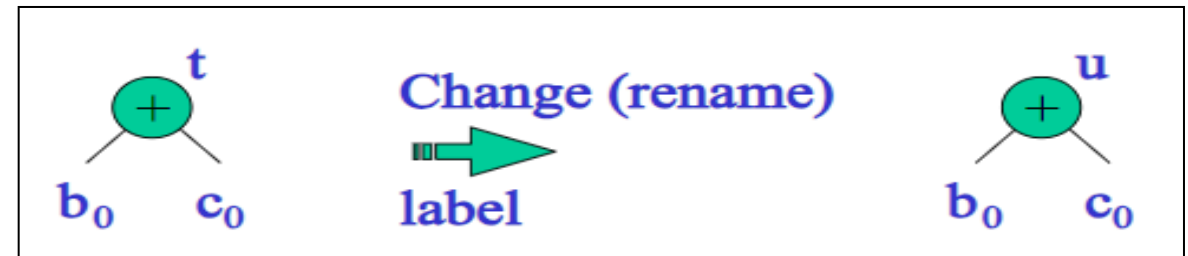
Commutative

- Say the operator * is commutative

$$\mathbf{x * y = y * x.}$$

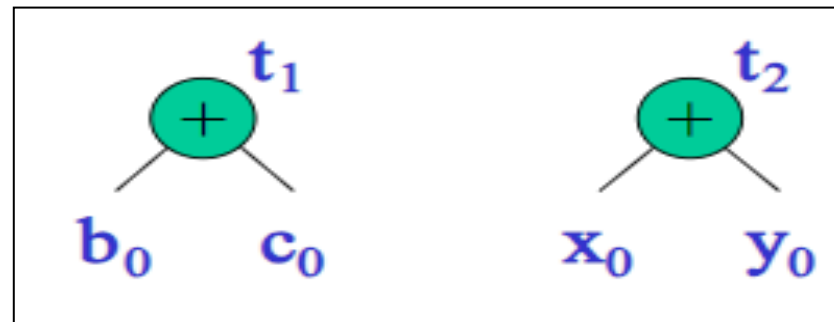
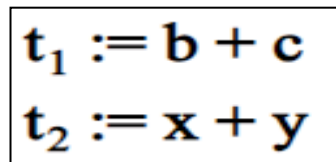
d. Renaming of temporary variables

- Statements containing instances of a temporary variable can be changed to instances of a new temporary variable without changing the basic block value.
- In this a basic block is transformed to its equivalent block called normal-form block.
- Consider the statement $t = b + c$



e. Interchange of two independent adjacent statements

- If a block has two adjacent statements which are independent can be interchanged without affecting the basic block value.



Self Study Topics

- **Representation of Array References**
- **Pointer Assignments and Procedure Calls**
- **Reassembling Basic Blocks From DAG's**